The merge sort Procedure consists of three steps:

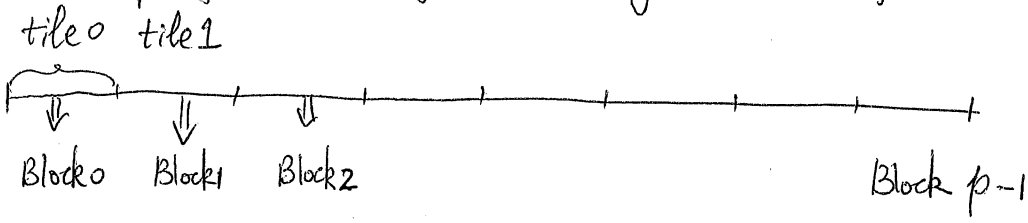
1) Divide the input into p equal-sized tiles.

Size of tile $\rightarrow t$; usually t is also the block size.

input data size $\rightarrow n$

$p \propto n/t \Rightarrow$ Assigning a single or small Constant number of input elements to each thread.

Research finds 256-thread block yields overall best performance for older generation of GPUs: GTX280, 8800 GT

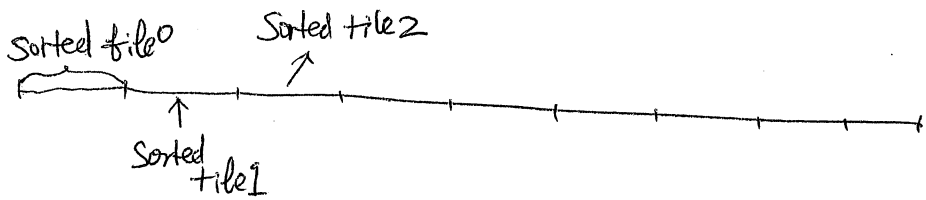


2) Sort all p tiles in parallel with p thread blocks

Each Block will sort an unique tile in the input data.

Block 0 process tile 0 ; Block 1 processes tile 1, ...

After this step is done, each tile (from 0 to tile p-1) is sorted within that tile.



We will study three different algorithms that sort an individual tile in parallel.

- naive merge kernel
- odd-even sorting network
- Advanced elegant merge kernel.

Binary Search Based

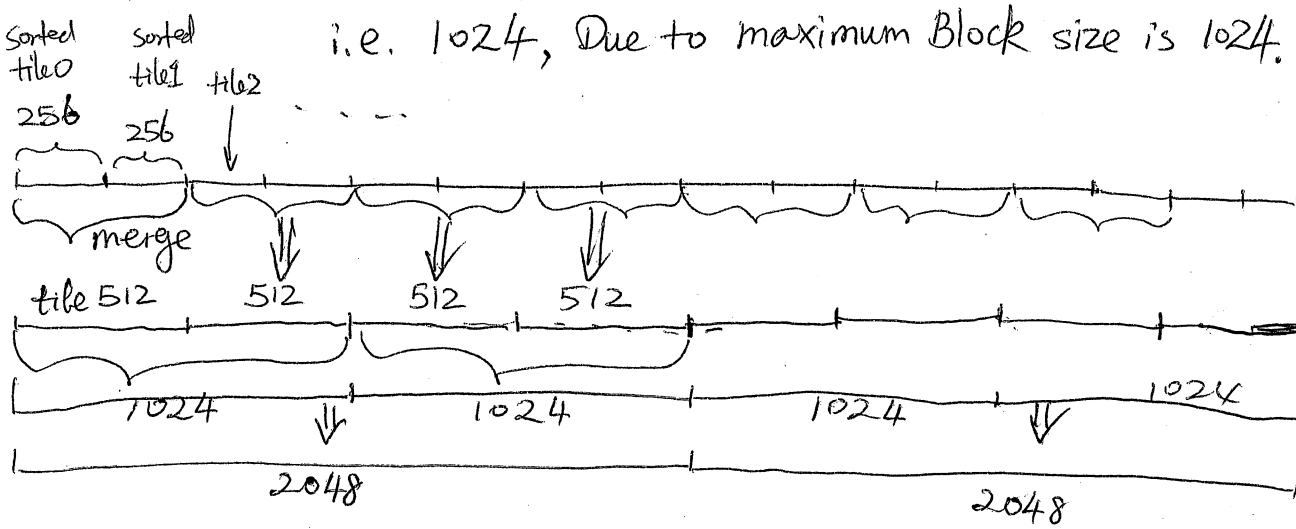
Merge Sort on CUDA device

Step 3) Merge all p sorted tiles

— it is easy for a thread block to merge two 256-element tiles into one 512-element tile.

— it is also fine for a thread block to merge two 512-element tiles into one 1024-element tile.

— Issue 1: If we use one thread to process one datum in a tile, there is a limit on the tile size, i.e. 1024, Due to maximum Block size is 1024.



✓ The sorted regions keep growing exponentially.

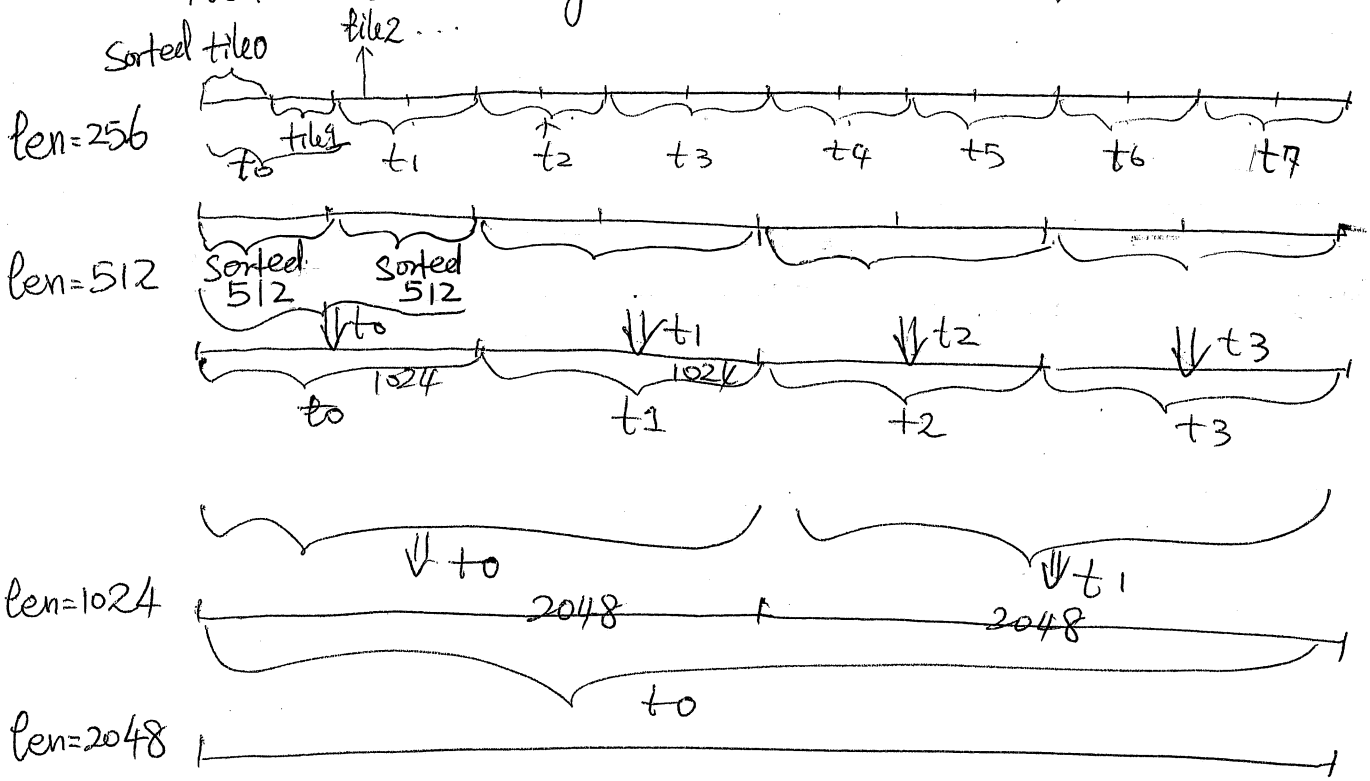
✓ If we use one thread to process one element during merging,

✓ The bigest region for a thread block to handle is ~~512~~ 1024.

ISSUE 2: If we adopt a naive merge procedure, similar to Naive merge kernel like the Naive merge sort kernel, each thread here in this naive merge tiles, will have to process multiple tiles. This Coarse-grained parallelism is not sufficient to fully utilize massively parallel architectures.

- We will talk about an elegant parallel merging approach.

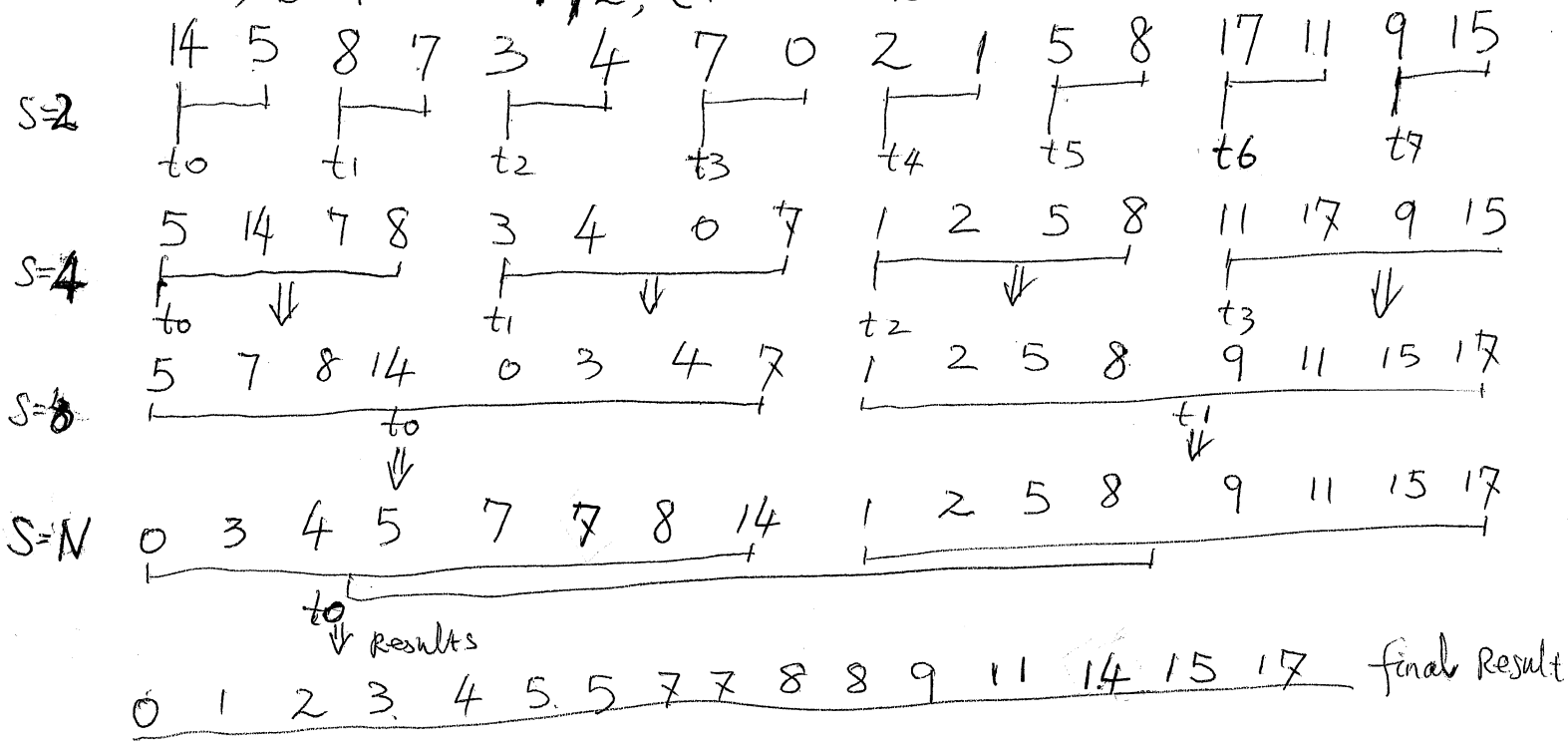
Naive tile merge on GPU; tile of 256 elements.



- each thead has to process "len" paires of elements in input. This is coarse-grain parallelism.
- Very inefficient!

naive merge sort kernel (sort a small chunk of data, with size smaller than Block size. This kernel sorts a tile of data)

- size of Tile is N;
- So, block size is N/2; (Thread block)



Observation: as kernel moving forward; Less and Less threads becomes active. At the end, only one thread does the merge operation.

So it is not efficient! The number of pairs to be merged decreases geometrically.